

Name: AKSHARAA BALASUBRMANIAN

Register Number: 192311278

Q1. Knapsack – Industry Resource Allocation A cloud service provider must allocate limited server memory (100 GB) among multiple client applications, each having different memory requirements and revenue values. Analyze the problem and explain how the 0/1 Knapsack algorithm can be applied to maximize total revenue. Design a feasible solution using dynamic programming and justify the selection of this approach. Discuss how overlapping subproblems and optimal substructure are utilized. Suggest suitable tools or platforms for implementation. Analyze the performance in terms of time and space complexity and interpret the results in an industry context.

## 1. Problem Analysis

A cloud service provider has a limited amount of **server memory (100 GB)**. Multiple client applications request different amounts of memory and generate different revenues.

The provider must decide **which applications to allocate memory to** such that:

- Each application is either **fully selected or not selected**.
- An application cannot be partially allocated.
- Total allocated memory must not exceed **100 GB**.
- Total revenue should be **maximum**.

This is a **0/1 Knapsack problem**, where:

- **Capacity (W)** = 100 GB
- **Weight** = Memory required by an application
- **Value** = Revenue generated by the application
- **Decision** = Select (1) or reject (0) each application

## 2. Why 0/1 Knapsack?

The 0/1 Knapsack algorithm is suitable because each client application has only two possibilities:

1 → Allocate memory to the application

0 → Do not allocate memory

Applications cannot be divided. Therefore, **fractional knapsack is not appropriate**.

Dynamic Programming is preferred because the same smaller allocation problems occur repeatedly while determining the optimal solution.

### 3. Example

Consider the following client applications:

| Application | Memory Required (GB) | Revenue |
|-------------|----------------------|---------|
| A1          | 20                   | 40      |
| A2          | 30                   | 60      |
| A3          | 40                   | 80      |
| A4          | 50                   | 90      |
| A5          | 60                   | 110     |

Server capacity = **100 GB**

A possible optimal selection is:

**A2 + A5**

- Memory =  $30 + 60 = \mathbf{90\ GB}$
- Revenue =  $60 + 110 = \mathbf{170}$

Another possibility:

**A1 + A2 + A3**

- Memory =  $20 + 30 + 40 = \mathbf{90\ GB}$
- Revenue =  $40 + 60 + 80 = \mathbf{180}$

Therefore, the second combination is better **Maximum Revenue = 180**

### 4. Dynamic Programming Approach

Create a DP table:

DP[i][w]

where:

- i = number of applications considered
- w = available memory from 0 to 100 GB

- $DP[i][w]$  = maximum revenue possible using the first  $i$  applications with capacity  $w$

For each application, there are two choices:

**Choice 1 – Do not select the application**

$$DP[i][w] = DP[i-1][w]$$

**Choice 2 – Select the application**

If its memory requirement is less than or equal to the available capacity:

$$DP[i][w] =$$

$$\max(\text{revenue}[i] + DP[i-1][w - \text{memory}[i]], DP[i-1][w])$$

Thus, the recurrence is:

$$DP[i][w] = \max(DP[i-1][w], \text{revenue}[i] + DP[i-1][w - \text{memory}[i]])$$

## 5. Pseudocode

Algorithm 0/1\_Knapsack(memory[], revenue[], n, W)

Create  $DP[0 \dots n][0 \dots W]$

For  $w = 0$  to  $W$

$$DP[0][w] = 0$$

For  $i = 1$  to  $n$

For  $w = 0$  to  $W$

If  $\text{memory}[i] \leq w$  then

$$DP[i][w] = \max(DP[i-1][w], \text{revenue}[i] + DP[i-1][w - \text{memory}[i]])$$

Else

$DP[i][w] = DP[i-1][w]$

Return  $DP[n][W]$

End Algorithm

For this problem:

$n = 5$

$W = 100$

memory = [20, 30, 40, 50, 60]

revenue = [40, 60, 80, 90, 110]

Maximum Revenue = 180

## 6. Overlapping Subproblems

Dynamic Programming avoids solving the same problem repeatedly.

For example, while calculating the best revenue for different applications, the algorithm may repeatedly need:

$DP[2][50]$

$DP[2][50]$

$DP[2][50]$

Instead of calculating it again each time, the result is **stored in the DP table** and reused.

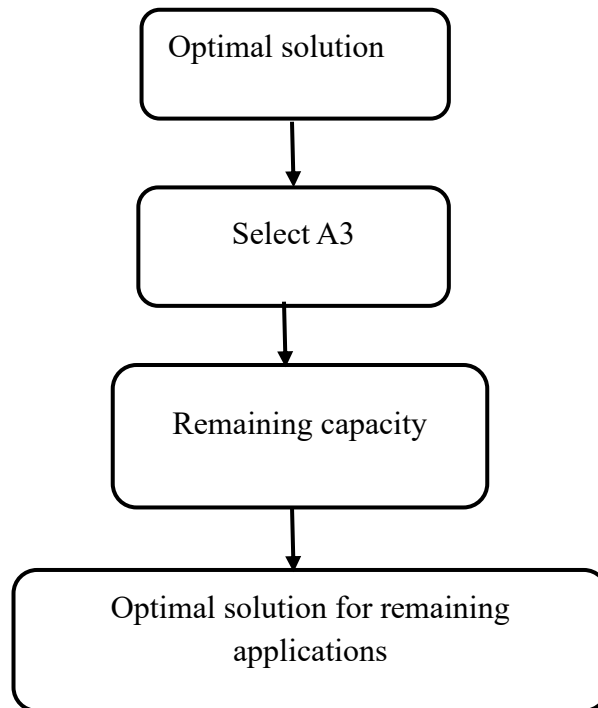
Therefore:

**Overlapping subproblems → Store previously calculated results → Avoid repeated computation**

## 7. Optimal Substructure

The problem has **optimal substructure** because the optimal solution for a larger problem contains optimal solutions to smaller problems.

For example, if application A3 is selected, the remaining capacity must be optimally allocated among the previous applications.



Thus, the overall optimal solution can be constructed from optimal solutions to smaller subproblems.

## 8. Why Dynamic Programming is Suitable

Dynamic Programming is suitable because:

- Memory capacity is limited.
- Each application has a binary selection decision.
- Applications cannot be split.
- Many subproblems overlap.
- Optimal substructure exists.
- It guarantees the **optimal revenue**, unlike simple greedy selection.

## 9. Suitable Implementation Tools

The solution can be implemented using:

| Tool/Platform    | Purpose                                    |
|------------------|--------------------------------------------|
| Python           | Easy implementation and testing            |
| Java             | Suitable for enterprise/cloud applications |
| C/C++            | High-performance implementation            |
| VS Code          | Coding and debugging                       |
| Jupyter Notebook | Algorithm demonstration and analysis       |
| Google Colab     | Cloud-based Python execution               |
| GitHub           | Version control and project management     |

## 10. Performance Analysis

Let:

- $n$  = number of client applications
- $W$  = server memory capacity

The DP table has  $(n + 1) \times (W + 1)$  entries.

### Time Complexity:

$O(n \times W)$

For 5 applications and 100 GB:

$$5 \times 100 = 500$$

approximately 500 DP states are evaluated.

### Space Complexity:

$O(n \times W)$

because the complete DP table is stored.

Space can also be optimized to:

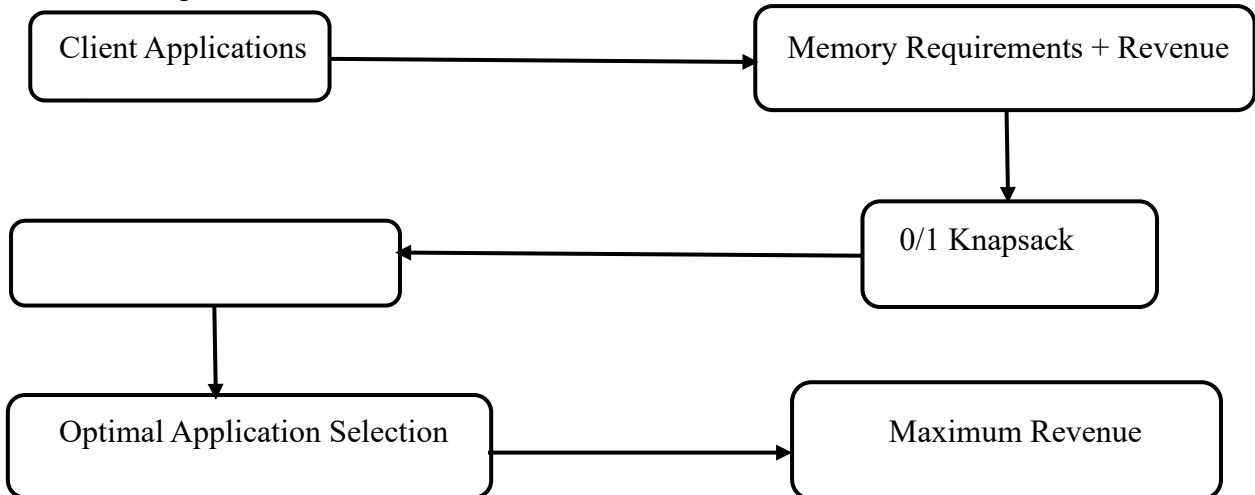
$O(W)$

by using a one-dimensional DP array.

## 11. Industry Interpretation

In a real cloud environment, the algorithm can help a provider decide which client workloads should receive limited resources when demand exceeds available capacity.

For example:



For the example, the provider can allocate resources to **A1, A2 and A3**, using **90 GB** and generating **180 units of revenue**.

This demonstrates how optimization algorithms can support **resource allocation, capacity planning, and revenue maximization** in cloud computing.

**Important industry limitation:** Real cloud systems usually involve additional factors such as CPU, network bandwidth, priorities, SLAs, scaling, and changing workloads. In such cases, the basic 0/1 Knapsack model may need to be extended to a **multi-dimensional or dynamic resource-allocation model**.

## 12. Gap Analysis

Traditional cloud resource allocation methods may allocate resources based on simple rules such as **First-Come-First-Serve, priority, or maximum revenue per GB**. These approaches do not always produce the globally optimal combination of applications.

### Identified Gap

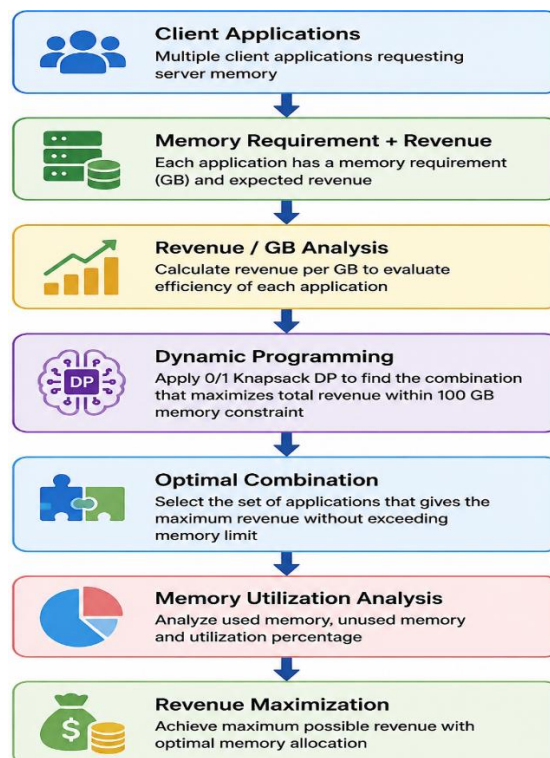
The major gap is that **simple resource allocation techniques do not guarantee maximum revenue under a strict memory constraint**.

For example, selecting the application with the highest revenue first does not necessarily produce the best overall combination. The provider needs an algorithm that evaluates **multiple combinations simultaneously** and identifies the globally optimal allocation.

| Existing Approach             | Gap / Limitation                                      | Proposed 0/1 Knapsack Solution                 |
|-------------------------------|-------------------------------------------------------|------------------------------------------------|
| First-Come-First-Serve        | May allocate memory to low-revenue applications first | Selects the combination that maximizes revenue |
| Greedy selection              | Locally optimal choice may not give global optimum    | DP guarantees the optimal combination          |
| Manual allocation             | Time-consuming and error-prone                        | Automated optimization                         |
| Fixed resource allocation     | Poor utilization when demand changes                  | Dynamically evaluates possible combinations    |
| Revenue-only decision         | May ignore memory constraints                         | Considers both memory and revenue              |
| Trial-and-error allocation    | Repeated calculations are required                    | Stores and reuses subproblem results           |
| Single-resource consideration | Real cloud systems have multiple resources            | Can be extended to CPU, storage and bandwidth  |

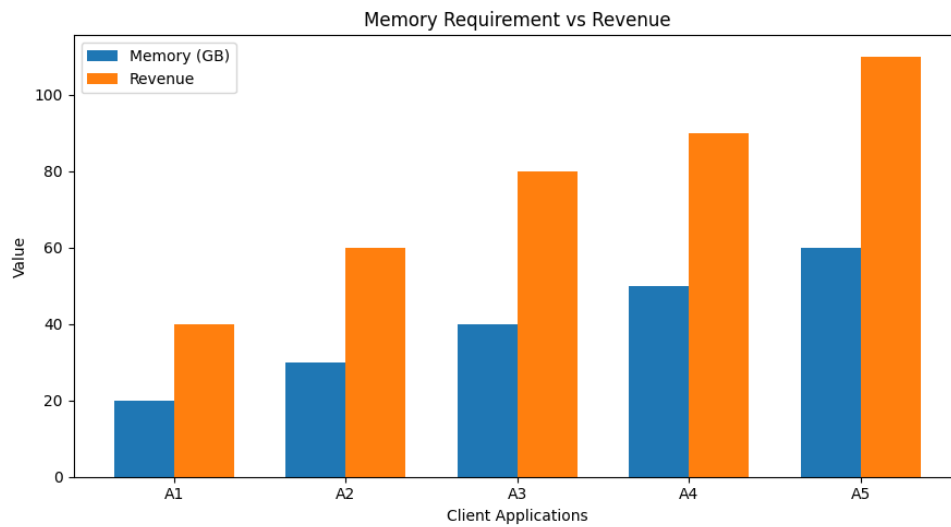
### 13. Unique Proposed Solution

Revenue-Aware Dynamic Cloud Resource Allocation : A unique extension of the basic 0/1 Knapsack approach is to develop a Revenue-Aware Cloud Resource Allocation System. Instead of simply calculating maximum revenue, the system can provide:



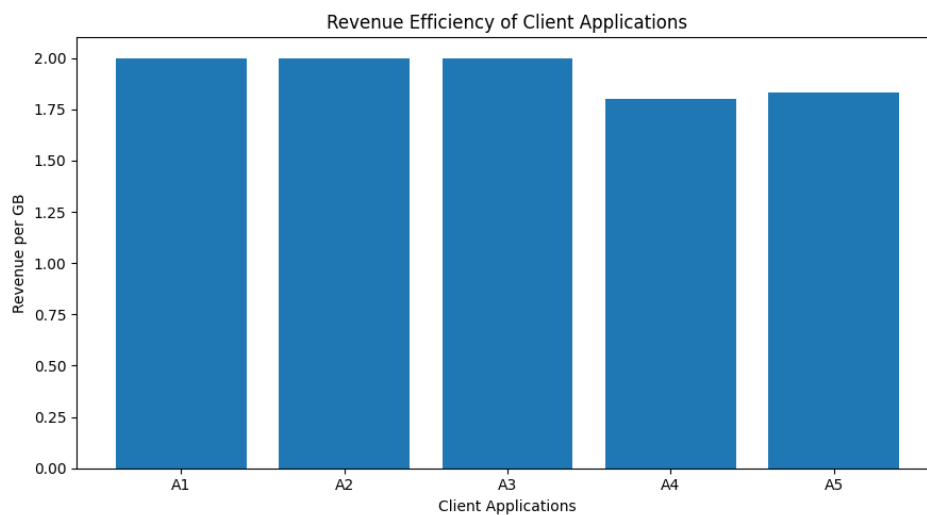


## 14. Interpretation



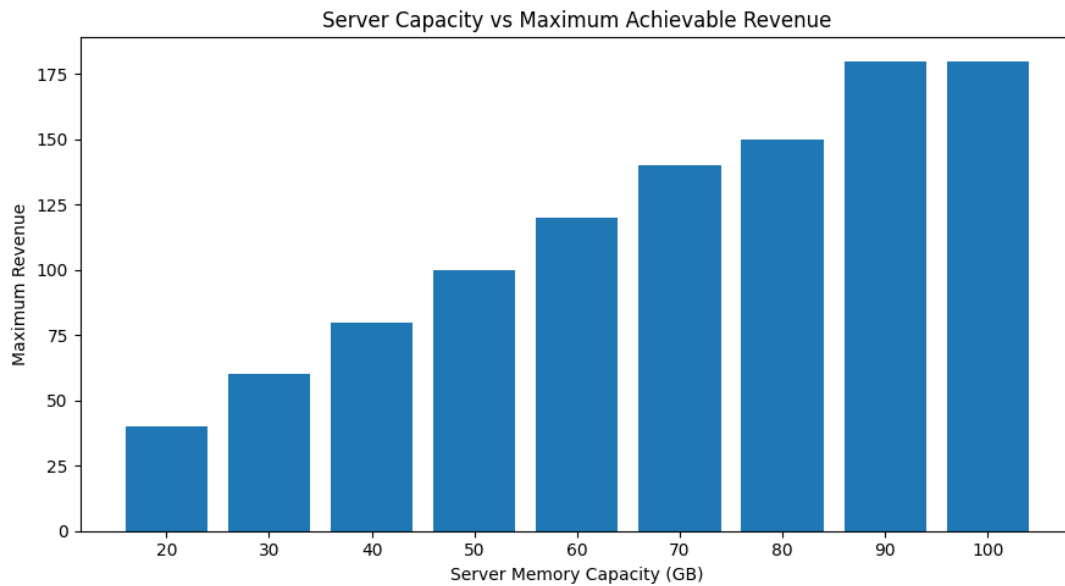
**Graph 1 – Memory vs Revenue:**

The graph compares the memory requirement and revenue contribution of each client application. Applications with larger memory requirements generally generate higher revenue, but selecting applications only based on individual revenue may not produce the optimal overall allocation.



**Graph 2 – Revenue per GB:**

Revenue per GB measures how efficiently each application uses server memory. A higher value indicates better revenue efficiency. However, this ratio is used only as an analytical indicator because a simple greedy selection based on revenue/GB cannot guarantee the globally optimal solution.



### Graph 3 – Capacity vs Maximum Revenue:

The graph demonstrates the relationship between available server memory and maximum achievable revenue. Revenue increases as capacity increases, but eventually reaches a point where additional memory does not provide additional revenue. In the example, revenue reaches **180 at 90 GB and remains 180 at 100 GB**, indicating that the additional 10 GB is not required for the current application set.

## 15. Conclusion

The **0/1 Knapsack algorithm using Dynamic Programming** provides an effective solution for allocating limited server memory among competing client applications while maximizing total revenue. By considering each application as either selected or rejected, the approach respects the indivisible nature of cloud resource allocation.

The use of **overlapping subproblems and optimal substructure** allows Dynamic Programming to avoid repeated calculations and guarantee the optimal allocation. The additional **Revenue/GB analysis, memory-utilization analysis, and capacity-versus-revenue graphs** provide useful insights beyond the basic algorithm.

In the example, the optimal combination achieves **maximum revenue of 180 units using 90 GB of the available 100 GB**, resulting in **90% memory utilization**. The capacity analysis also shows that increasing capacity beyond the required level does not always increase revenue. Therefore, this approach can help cloud providers make **data-driven resource allocation decisions, maximize revenue, reduce resource wastage, and support capacity planning**. In a real-world cloud environment, the model can be further enhanced by incorporating **dynamic pricing, CPU and storage constraints, workload priority, SLA requirements, and real-time resource availability**.